

Nautilus — Requirement Analysis

Software Group Project · Reporting 2

1. Purpose

Turn a folder of personal notes into a system that can show what you know (dashboard), answer questions about it (chat bot), and maintain itself (scheduled AI tasks).

2. Functional Requirements — what the system must do

ID	Requirement — and what it means in practice
FR1	Read all markdown notes from a chosen folder (the vault). Folders inside it act as domains — this is how notes get grouped without any setup.
FR2	List every note with its title, domain, and last-modified time — powers the 'recent notes' panel.
FR3	Count notes per domain and in total — powers the orbit visualization and stat cards.
FR4	Detect [[wikilinks]] between notes and report the link graph, including broken links — shows how connected the knowledge is.
FR5	Answer a typed question using the user's own notes: find relevant notes, send them with the question to an LLM, return the answer.
FR6	Name the source notes used in every answer — the user can always verify where an answer came from.
FR7	Show domains as an orbit around a central count, sized by how many notes each holds — the visual identity of the product.
FR8	Clicking a domain filters the note list and pre-fills a question about it — one tap from overview to conversation.
FR9	Auto-organize: a nightly scheduled AI task files rough daily captures into the right domain with tags and links. (via Cowork)
FR10	Weekly digest: a scheduled task writes a summary note of the week into the vault every Monday. (via Cowork)
FR11	Conflict check: a weekly task flags notes that contradict each other, with links to both. (via Cowork)

3. Non-Functional Requirements — how it must behave

Quality	Requirement — and why it matters
Local-first	Everything runs on the user's machine; note text leaves it only as context for an LLM query. Personal knowledge stays personal.
Always in sync	No database — the vault is read fresh from disk on every request, so the dashboard can never show stale data.
Simple to run	Frontend is one HTML file (double-click to open); backend starts with one command. No build tools, no installers.
Low cost	Runs on a free LLM tier for personal volumes; automation is covered by an existing Claude subscription.
No lock-in	Notes stay plain markdown, fully usable in Obsidian or any editor. Deleting Nautilus loses nothing.
Swappable AI	The LLM provider is isolated to one function — switching models is a minutes-long change, not a rewrite.

4. Assumptions & Constraints

- Single user, personal vault, running locally (public hosting is out of scope for v1).
- Note retrieval uses keyword scoring — sufficient for hundreds of notes; semantic search is a planned upgrade.
- Scheduled automation requires a paid Claude plan (Cowork); the rest of the system runs without it.